

TASK 1 – REACT BASICS

1. What is React?

React is a JavaScript library used to build user interfaces (UI), especially for web applications. It was developed by Meta (Facebook). React helps us create interactive web pages using reusable components.

2. Why do we use React?

We use React because:

- **It helps build interactive user interfaces.**
- **We can divide the UI into reusable components.**
- **It makes large applications easier to manage.**
- **It efficiently updates the UI when data changes.**
- **It supports Single Page Applications (SPA).**
- **It allows us to use JavaScript and JSX together.**

3. Is React a library or framework?

React is a JavaScript library, not a complete framework.

React mainly focuses on building the UI.

For routing, state management, API handling, etc., we can use additional libraries or tools.

4. What are the main features of React?

Main features of React are:

- 1. Components – Build UI using small reusable parts.**
- 2. JSX – Write HTML-like syntax inside JavaScript.**
- 3. Virtual DOM – Helps React efficiently update the UI.**
- 4. Declarative UI – We describe what the UI should look like.**
- 5. One-way data flow – Data generally flows from parent to child.**
- 6. Reusable components – Components can be used multiple times.**
- 7. State and Props – Used to manage and pass data.**

5. What is a Single Page Application (SPA)?

SPA stands for Single Page Application.

It is a web application where the browser initially loads one main HTML page, and React updates the required parts of the page without completely reloading the page for every interaction.

6. What is declarative UI?

Declarative UI means we describe what the UI should look like, and React handles how to update the DOM.

TASK 2 – COMPONENTS

1. What is a Component in React?

A component is a reusable building block of a React application.

A component usually contains:

- **UI**
- **JavaScript logic**
- **Data handling**

2. Why do we use Components?

We use components to:

- **Divide a large UI into smaller parts.**
- **Reuse the same UI.**
- **Make code easier to understand.**
- **Make applications easier to maintain.**
- **Separate different parts of the application.**

3. What is a Functional Component?

A Functional Component is a JavaScript function that returns JSX.

Example:

```
function Welcome() {  
  
  return <h1>Welcome</h1>;  
  
}
```

Here, `Welcome()` is a functional component.

4. What is Component-Based Architecture?

Component-Based Architecture means building an application by dividing it into smaller independent and reusable components.

5. What is a Component Tree?

A Component Tree represents the parent-child relationship between React components.

6. Why should component names start with a capital letter?

React uses the capital letter to distinguish a user-defined component from a normal HTML element.

7. What is a reusable component?

A reusable component is a component that can be used multiple times with different data.

TASK 3 – JSX

1. What is JSX?

JSX stands for JavaScript XML.

It is a syntax extension for JavaScript that allows us to write HTML-like code inside JavaScript.

2. Why do we use JSX in React?

We use JSX because it makes UI code:

- **Easier to read.**
- **Easier to write.**
- **Easier to understand.**
- **Easier to combine with JavaScript logic.**

3. Is JSX HTML?

No. JSX is not HTML.

JSX looks similar to HTML, but it is a JavaScript syntax extension.

4. How do we write JavaScript expressions inside JSX?

We use curly braces `{}`.

Example:

```
const name = "Subashini";  
return <h1>Hello {name}</h1>;
```

Output:

Hello Subashini

We can also write expressions:

```
<p>{10 + 20}</p>
```

5. Why do we use `className` instead of `class`?

In JSX, we use:

`className`

instead of:

`class`

Example:

```
<div className="container">
```

Hello

```
</div>
```

`class` is a JavaScript-related reserved word, so JSX uses `className` for CSS classes.

6. What is React Fragment?

A React Fragment allows us to group multiple elements without adding an extra HTML element to the DOM.

7. What are the important JSX rules?

Important JSX rules:

1. Return one parent element

2. Close all tags

3. Use `className`

```
<div className="box"></div>
```

4. Use `{ }` for JavaScript expressions

```
<h1>{name}</h1>
```

5. Component names start with capital letters

```
<Header />
```